Visual AI BDD Framework

Complete Setup & Execution Guide

PROJECT: Visual AI BDD Framework using Selenium + Applitools + Cucumber + JUnit + Allure

PURPOSE: Automated visual regression testing — detects UI changes, layout breaks, and visual bugs on websites

SUBMISSION: Open this project in IntelliJ IDEA and follow each step in order. No prior coding knowledge needed.

Section 1 — What You Need to Install First

Before running the project, install the following software on your computer. Each item is free to download.

1.1 Java Development Kit (JDK) 11 or higher

- Download from: https://adoptium.net (choose JDK 11 LTS)
- Run the installer and follow the steps
- After installing, open Command Prompt and type: java -version
- You should see something like: openjdk version 11.x.x

1.2 Apache Maven

- Download from: https://maven.apache.org/download.cgi
- Extract the ZIP file to a folder (e.g., C:\Program Files\Maven)
- Add Maven's bin folder to your system PATH environment variable
- Open Command Prompt and type: mvn -version — you should see Maven 3.x.x

TIP: If you are not sure how to set PATH, search Google for: 'How to add Maven to PATH on Windows/Mac'

1.3 Google Chrome Browser

- Download from: https://www.google.com/chrome
- Install normally — this is the browser the tests will run on

1.4 IntelliJ IDEA (already on your computer from the pendrive)

- Download Community Edition (free) from: https://www.jetbrains.com/idea/download
- Install and open IntelliJ IDEA

Section 2 — Opening the Project in IntelliJ IDEA

STEP 1

Copy the project folder

Copy the 'coding' folder from the pendrive to your Desktop or Documents folder (e.g., C:\Users\YourName\Desktop\coding)

STEP 2

Open IntelliJ IDEA

Launch IntelliJ IDEA. On the Welcome screen, click 'Open'. Browse to the 'coding' folder you copied and click OK.

STEP 3

Let Maven download dependencies

IntelliJ will show a notification at the bottom: 'Maven projects need to be imported'. Click 'Load Maven Project'. Wait for the download bar at the bottom to finish (this may take 3-5 minutes on first open).

STEP 4

Confirm Java SDK is set

Go to: File → Project Structure → Project. Make sure 'SDK' shows Java 11 or higher. If empty, click New → JDK and point to where you installed JDK.

IMPORTANT: Do NOT move or rename the project folder after opening it in IntelliJ. This can break the project paths.

Section 3 — Configuring the Project

The project has one configuration file you need to check before running.

3.1 Open framework.properties

In the IntelliJ left panel, navigate to:

```
coding/src/main/resources/framework.properties
```

The file looks like this:

```
applitools.api.key=JqDmQo0sdN98sZhEx0dkTaTZQGgvEh3Nicl4sFu6CzA110
applitools.app.name=VisualAI-BDD-Framework
applitools.match.level=LAYOUT
```

```
browser=chrome
browser.headless=false

viewport.width=1366
viewport.height=768

visual.full.page=true
page.load.timeout=30

report.cucumber.output=target/cucumber-reports
report.allure.results=target/allure-results
```

NOTE: The Applitools API key is already filled in. You do NOT need to change it. This key connects to the Applitools cloud for visual comparison.

3.2 Settings you can change

Setting	Default value	What it does
`browser`	`chrome`	Which browser opens (chrome / firefox / edge)
`browser.headless`	`false`	false = browser window opens | true = runs in background
`applitools.match.level`	`LAYOUT`	How strictly visuals are compared (LAYOUT / STRICT / EXACT)
`viewport.width / height`	`1366 x 768`	Browser window size during testing
`page.load.timeout`	`30`	Seconds to wait for page to load before giving up

Section 4 — Running the Tests

There are TWO ways to run the tests. Use Method A (IntelliJ Terminal) — it is the easiest.

Method A — Using the IntelliJ Terminal (Recommended)

1. In IntelliJ, click 'Terminal' at the bottom of the screen (or press Alt+F12)
2. Make sure the terminal shows the path ending in: ...\coding>
3. Type the following command and press Enter:

```
mvn clean test -Dwebsite=https://www.google.com
```

Replace https://www.google.com with the website you want to test.

EXAMPLE WEBSITES YOU CAN TEST: https://www.google.com | https://www.github.com | https://www.wikipedia.org

What happens after you press Enter:

- Maven downloads any missing files (first time only — internet needed)
- Chrome browser opens automatically
- The website loads in Chrome
- Screenshots are taken and sent to Applitools for comparison
- Chrome closes after each test
- You will see BUILD SUCCESS or BUILD FAILURE in the terminal

Method B — Using the Run Configuration in IntelliJ

4. In IntelliJ, go to: Run → Edit Configurations
5. Click the + button and choose 'Maven'
6. In the 'Command line' field type: clean test -Dwebsite=https://www.google.com
7. Click OK, then click the green Play button at the top right

Section 5 — Understanding the Test Results

5.1 Terminal output — what to look for

```
Tests run: 14, Failures: 1, Errors: 2, Skipped: 0

BUILD SUCCESS   ← all tests passed (failures are intentional in negative
tests)
BUILD FAILURE   ← something crashed or a required setting is missing
```

NOTE: This framework has some INTENTIONAL failures to demonstrate what a visual defect looks like. BUILD SUCCESS is expected even with some test failures.

5.2 Viewing the Cucumber HTML Report

- After the run, go to the folder: target/cucumber-reports/
- Open the file: cucumber-native.html in your web browser
- This shows a full report of every scenario: PASSED, FAILED, or ERROR
- Green = passed, Red = failed, Yellow = error

5.3 Viewing the Allure Report

Allure gives a richer, more visual report. To open it:

```
# In the IntelliJ Terminal, type:
.allure\allure-2.24.0\bin\allure serve target\allure-results

# On Mac/Linux:
.allure/allure-2.24.0/bin/allure serve target/allure-results
```

- A browser window will open automatically with the Allure dashboard
- You can see test timelines, per-scenario details, and attachment logs

5.4 Viewing results on Applitools Dashboard

- Go to: https://eyes.applitools.com
- Log in with the credentials linked to the API key
- You will see baseline vs actual screenshots side by side
- Differences are highlighted automatically

Section 6 — Running Only Specific Tests

The project has multiple test scenarios grouped by tags. You can run just the ones you want.

Tag / Command	What it runs	Expected result
`@smoke`	Quick full-page layout check only	Should PASS
`@region`	Validates header, footer, nav sections	Should PASS
`@responsive`	Tests across 4 screen sizes	Intentional ERROR (demo)
`@negative`	Injects visual defects to detect	Intentional behavior
`@missing`	Removes UI sections to test detection	Mix of PASS/FAIL
`@strict`	Pixel-level comparison mode	Should PASS

Command to run a specific tag:

```
1. mvn clean test -Dwebsite=https://www.google.com

2. mvn clean test -Dwebsite=https://www.google.com -
   Dcucumber.filter.tags="@smoke"
```

Section 7 — Common Problems and Fixes

Problem: 'java' is not recognized as a command

- Java is not installed or not in your PATH
- Fix: Install JDK from https://adoptium.net and restart your computer

Problem: 'mvn' is not recognized as a command

- Maven is not installed or not in your PATH
- Fix: Install Maven and add its bin folder to the PATH variable

Problem: ChromeDriver error / Chrome not found

- Google Chrome is not installed
- Fix: Install Chrome from https://www.google.com/chrome
- Note: The framework auto-downloads the matching ChromeDriver — you do NOT need to install it manually

Problem: 'No website provided via -Dwebsite'

- You forgot to add the -Dwebsite=... part to the command
- Fix: Always include -Dwebsite=https://yoursite.com in the command

Problem: Tests run but all fail instantly

- The website you are testing may be blocking automated browsers
- Fix: Try a different website like https://www.wikipedia.org

Problem: Maven downloads never finish

- Check your internet connection
- If on a company or college network, Maven may be blocked by a firewall
- Fix: Try on a different network (e.g., mobile hotspot)

Section 8 — Project Structure Explained

```
coding/
  src/
    main/java/com/visualai/
      config/FrameworkConfig.java        ← reads framework.properties
      driver/DriverFactory.java           ← opens/closes the browser
      stepdefinitions/VisualValidationSteps.java  ← what each step does
      utils/WebsiteInputResolver.java     ← reads the -Dwebsite value
      visual/VisualTestManager.java       ← talks to Applitools API
```

```
    main/resources/
      framework.properties              ← ALL settings live here
    test/java/com/visualai/runner/
      TestRunner.java                   ← entry point that starts
everything
    test/resources/features/
      visual_validation.feature         ← all test scenarios written in
plain English
  target/                               ← generated after running
(reports are here)
  pom.xml                               ← list of all libraries used
```

Quick Reference — Commands

```
# Run ALL tests on a website:
mvn clean test -Dwebsite=https://www.google.com

# Run ONLY smoke tests:
mvn clean test -Dwebsite=https://www.google.com -
Dcucumber.filter.tags="@smoke"

# Run ONLY region tests:
mvn clean test -Dwebsite=https://www.google.com -
Dcucumber.filter.tags="@region"

# Run with headless browser (no visible Chrome window):
# Edit framework.properties → set: browser.headless=true
# Then run: mvn clean test -Dwebsite=https://www.google.com

# Open Allure report after tests:
.allure\allure-2.24.0\bin\allure serve target\allure-results

# Clean previous results before re-running:
mvn clean
```

REMEMBER: Always run from the 'coding' folder in the terminal. If the terminal prompt does not end with ...\coding>, type: cd path\to\coding then press Enter.